Diamond Dogs

Comparativa de librerías de regresión lineal: scikit-learn y NumPy

Criterio	scikit-learn	NumPy
Facilidad de uso	Fácil: crear el modelo, entrenar (`fit`) y predecir (`predict`)	Fácil para la simple; la múltiple, un poco más difícil (pasos manuales)
Documentación y comunidad	Guía de usuario con muchos ejemplos; comunidad muy grande	Documentación oficial amplia, pero no centrada en regresión; comunidad muy grande
Regresión simple	`LinearRegression` con una columna	`polyfit(x, y, 1)`
Regresión múltiple	La misma clase con varias columnas	`linalg.lstsq` con una columna de unos
Coeficientes y término independiente	`coef_` e `intercept_`	El array devuelto (hay que saber qué posición es cada uno)
Métricas de error	`mean_squared_error` y `r2_score`	No incluye; se calculan a mano
Instalación con uv	`uv add scikit-learn`	Ya instalada con pandas; `uv add numpy`

Explicación de los criterios

Facilidad de uso. scikit-learn sigue siempre los mismos pasos: crear el modelo, entrenarlo con `fit` y predecir con `predict`. Con NumPy la regresión simple es una sola línea, pero la múltiple requiere preparar a mano la matriz de entradas, lo que la hace un poco más difícil.

Documentación y comunidad. Las dos tienen documentación oficial y una comunidad muy grande. La de scikit-learn incluye una guía de usuario con muchos ejemplos de regresión pensados para principiantes; la de NumPy es general y no está centrada en regresión.

Regresión simple y múltiple. En scikit-learn el código es el mismo para las dos: solo cambia el número de columnas de entrada. En NumPy, `polyfit` solo admite una entrada; para varias hay que usar `linalg.lstsq` y añadir una columna de unos para obtener el término independiente.

Coeficientes y término independiente. scikit-learn los da por separado en `coef_` e `intercept_`, lo que facilita escribir la fórmula del modelo. NumPy devuelve un array y hay que saber qué posición corresponde a cada término.

Métricas de error. scikit-learn incluye `mean_squared_error` (ECM) y `r2_score` (R^2), que sirven igual para entrenamiento y para test. NumPy no incluye métricas: se calculan con sus fórmulas.

Instalación con uv. scikit-learn: `uv add scikit-learn`. NumPy ya está instalada porque pandas depende de ella.

Elección para Diamond Dogs

Se elige scikit-learn. El mismo código sirve para regresión simple y múltiple, trabaja directamente con los DataFrame que devuelve `separar_datos`, da los coeficientes y el término independiente por separado e incluye las métricas ECM y R^2, que con NumPy habría que programar a mano. Además, ya era una dependencia del proyecto, porque `separar_datos` usa su función `train_test_split`.

Prueba de concepto

El script `prototipos/prueba_regresion.py`, sin interfaz gráfica, se ejecuta con `uv run python prototipos/prueba_regresion.py`. Usa el dataset `data/housing.csv` (20.640 filas; cada fila es un distrito de California) y reutiliza el código de la aplicación:

1. Carga los datos con `ImportadorDatos`.

2. Separa los datos en entrenamiento (80 %) y test (20 %) con `separar_datos` y la semilla 42. Las filas con valores inexistentes se eliminan con `eliminar_nulos`, porque `LinearRegression` no los acepta.

3. Entrena `LinearRegression` con el conjunto de entrenamiento.

4. Obtiene `coef_` e `intercept_` y escribe la fórmula con los nombres de las columnas.

5. Calcula el ECM y el R^2 en entrenamiento y en test.

6. Predice el valor para una entrada nueva.

Resultados

Se probó un modelo de regresión múltiple con las entradas `median_income`, `housing_median_age` y `total_rooms`, y la salida `median_house_value` (valor mediano de las viviendas del distrito, en dólares). Fórmula obtenida (salida del script):

```
median_house_value = 42653.33 * median_income + 1975.74 * housing_median_age
                   + 3.85 * total_rooms - 25029.51
```

Métrica	Entrenamiento	Test
Filas	16.512	4.128
ECM	6.453.767.350,33	6.563.178.660,01
Raíz del ECM	80.335,34 $	81.013,45 $
R^2	0,5172	0,4992

Predicción para una entrada nueva (`median_income` = 5,0; `housing_median_age` = 20; `total_rooms` = 2.000): **235.454,09 $**.

Interpretación de los resultados

Coeficientes. Por cada punto más de `median_income` (10.000 $ más de ingresos medios) el valor predicho sube 42.653,33 $, manteniendo iguales las demás entradas. Cada año más de `housing_median_age` lo sube 1.975,74 $, posiblemente porque los distritos más antiguos están en zonas céntricas o costeras. `total_rooms` apenas influye (3,85 $ por habitación) porque es el total de habitaciones de todo el distrito.

Término independiente. Es el valor predicho cuando todas las entradas valen 0 (-25.029,51 $). No tiene sentido real, porque no existen distritos así; solo sirve para colocar el modelo cerca de los datos.

ECM. Es la media de los errores al cuadrado; por eso está en dólares al cuadrado y sus valores son tan grandes. Su raíz devuelve las unidades originales: el modelo se equivoca de forma típica en unos 81.013 $ en test. Es un error alto: para unas viviendas que valen 200.000 $, el modelo podría predecir unos 120.000 $ o unos 280.000 $.

R^2. Es un número, normalmente entre 0 y 1, que mide lo bien que encajan las predicciones del modelo con los valores reales: 1 significa que acierta todo y 0, que no sirve para predecir. El 0,4992 de test significa que las tres entradas explican aproximadamente la mitad de por qué unos distritos son más caros que otros; la otra mitad depende de cosas que el modelo no conoce, como la ubicación.

Entrenamiento frente a test. El R^2 es muy parecido en los dos conjuntos (0,5172 frente a 0,4992). Esto demuestra que el modelo ha aprendido de verdad una regla general y no ha memorizado los datos de entrenamiento: funciona casi igual con datos que nunca ha visto. Que test sea algo peor es lo esperable.

Dificultades encontradas

Regresión múltiple y métricas con NumPy. Hacer la regresión múltiple con NumPy resultó un poco más difícil que con scikit-learn: hay que añadir a mano una columna de unos para obtener el término independiente y saber qué posición del resultado corresponde a cada coeficiente. Además, NumPy no incluye funciones para las métricas, así que el ECM y el R^2 hay que calcularlos a mano con sus fórmulas.

Valores inexistentes. `LinearRegression` no acepta NaN y algunas columnas del dataset, como `total_bedrooms`, tienen celdas vacías. Se resolvió reutilizando `eliminar_nulos` del preprocesado antes de entrenar.

Formato de los resultados. Como la salida se pasa como DataFrame, `coef_`, `intercept_` y `predict` devuelven los valores dentro de una lista adicional; se accede a ellos con `[0]` (`[0][0]` en la predicción).

Interpretar el ECM. Sus valores eran difíciles de entender por estar en unidades al cuadrado; se calculó su raíz para expresar el error en dólares.

Ruta del dataset. Una ruta relativa como `data/housing.csv` depende de la carpeta desde la que se ejecuta el script; se construyó a partir de la ubicación del propio script con `Path(__file__)`.

Fórmula legible. Para mostrar los coeficientes negativos con el signo correcto ("- 25029.51" en lugar de "+ -25029.51") se escribió la función `escribir_formula`.

El prototipo se guarda en `prototipos/`, separado del código de la aplicación en `src/`. scikit-learn queda añadida como dependencia del proyecto con `uv add scikit-learn`.